

RE Leçon 3 — Assembleur x86/x64

Pourquoi lire l'assembleur ?

En RE, on n'a jamais le code source. On a uniquement le binaire compilé. GDB et Ghidra le désassemblent en assembleur. Savoir le lire = comprendre ce que fait le programme.

Code C (invisible)		Assembleur (ce qu'on voit)
int x = 5;	→	mov DWORD PTR [rbp-0x4], 0x5
if (x > 3)	→	cmp DWORD PTR [rbp-0x4], 0x3
printf("oui")	→	jle .L2
	→	call printf

1. Les registres x64

Les registres sont les "variables" du processeur — ultra-rapides, en nombre limité.

Registres généraux

64 bits	32 bits	16 bits	8 bits	Rôle principal
rax	eax	ax	al	Valeur de retour, résultats
rbx	ebx	bx	bl	Base, usage général
rcx	ecx	cx	cl	Compteur de boucle
rdx	edx	dx	dl	Données, 3ème argument
rsi	esi	si	sil	2ème argument
rdi	edi	di	dil	1er argument
r8-r15	r8d-r15d	—	—	Arguments 5-6, usage général

Registres spéciaux

Registre	Rôle
rsp	Stack Pointer — sommet de la pile
rbp	Base Pointer — base du stack frame courant
rip	Instruction Pointer — adresse de l'instruction courante

Convention d'appel x64 (Linux)

Les 6 premiers arguments d'une fonction sont passés dans :

```
1er → rdi
2ème → rsi
3ème → rdx
4ème → rcx
5ème → r8
6ème → r9
```

La valeur de retour est dans **rax**.

2. Instructions de base

mov — déplacer des données

```
mov rax, 42          ; rax = 42
mov rax, rbx         ; rax = rbx
mov rax, [rbp-0x4]   ; rax = valeur en mémoire à l'adresse rbp-4
mov [rbp-0x4], rax   ; mémoire à rbp-4 = rax
```

[...] = accès à la mémoire à cette adresse (comme ***p** en C)

push / pop — la pile

```
push rax    ; empile rax (rsp -= 8, puis *rsp = rax)
pop rbx     ; dépile dans rbx (rbx = *rsp, puis rsp += 8)
```

add / sub — arithmétique

```
add rax, 5      ; rax = rax + 5
sub rsp, 0x10   ; rsp = rsp - 16 (réserver de la place sur la pile)
```

cmp — comparaison

```
cmp rax, 5      ; calcule rax - 5 et met à jour les flags (sans stocker)
```

Seul, **cmp** ne fait rien de visible. Il est toujours suivi d'un saut conditionnel.

Sauts conditionnels (après **cmp**)

Instruction	Signification	Équivalent C
<code>je / jz</code>	Jump if Equal	<code>if (a == b)</code>
<code>jne / jnz</code>	Jump if Not Equal	<code>if (a != b)</code>
<code>jl / jlt</code>	Jump if Less	<code>if (a < b)</code>
<code>jle</code>	Jump if Less or Equal	<code>if (a <= b)</code>
<code>ja / jgt</code>	Jump if Greater	<code>if (a > b)</code>
<code>jge</code>	Jump if Greater or Equal	<code>if (a >= b)</code>
<code>jmp</code>	Jump inconditionnel	<code>goto</code>

call / ret — appel de fonction

```
call printf    ; empile l'adresse de retour, saute à printf
ret           ; dépile l'adresse de retour, saute dessus
```

3. Le Stack Frame

Chaque appel de fonction crée un **stack frame** — une zone sur la pile qui contient les variables locales et l'adresse de retour.

```
Prologue (début de fonction) :
    push rbp          ; sauvegarder l'ancien rbp
    mov rbp, rsp      ; nouveau frame
    sub rsp, 0x20     ; réserver 32 octets pour les variables locales

Épilogue (fin de fonction) :
    leave             ; = mov rsp, rbp + pop rbp
    ret              ; retourner à l'appelant
```

Variables locales en mémoire :

```
rbp-0x4 → 1ère variable locale (int)
rbp-0x8 → 2ème variable locale (int)
rbp-0x10 → 3ème variable locale (int)
...
```

4. Reconnaître les patterns C en assembleur

Pattern — variable locale

```
int x = 42;
```

```
mov DWORD PTR [rbp-0x4], 42
```

Pattern — if / else

```
if (x > 10) {  
    printf("grand\n");  
} else {  
    printf("petit\n");  
}
```

```
cmp DWORD PTR [rbp-0x4], 10    ; compare x à 10  
jle .else                     ; si x <= 10 → aller à else  
lea rdi, [msg_grand]  
call printf  
jmp .fin  
.else:  
lea rdi, [msg_petit]  
call printf  
.fin:
```

Astuce RE : un `cmp` suivi d'un saut conditionnel = structure if/else.

Pattern — boucle for

```
for (i = 0; i < 5; i++) {  
    printf("%d\n", i);  
}
```

```
mov DWORD PTR [rbp-0x4], 0    ; i = 0  
.debut_boucle:  
cmp DWORD PTR [rbp-0x4], 5    ; i < 5 ?  
jge .fin_boucle               ; si i >= 5 → sortir  
mov esi, DWORD PTR [rbp-0x4]  ; argument : i  
lea rdi, [format]  
call printf  
add DWORD PTR [rbp-0x4], 1    ; i++
```

```
jmp .debut_boucle
.fin_boucle:
```

Astuce RE : `jmp` vers une adresse précédente + `cmp/jge` = boucle.

Pattern — appel de fonction

```
int z = additionner(3, 5);
```

```
mov esi, 5          ; 2ème argument
mov edi, 3          ; 1er argument
call additionner
mov DWORD PTR [rbp-0x4], eax ; z = valeur retournée (dans rax)
```

Pattern — accès tableau

```
int tab[5] = {1, 2, 3, 4, 5};
int val = tab[2];
```

```
; tab est à rbp-0x20
mov eax, DWORD PTR [rbp-0x20 + 2*4] ; tab[2] = base + index*4
; ou
lea rax, [rbp-0x20] ; rax = adresse de tab
mov eax, DWORD PTR [rax+0x8] ; [rax + 2*4]
```

Pattern — appel strcmp (crackme classique)

```
if (strcmp(input, "secret") == 0)
```

```
lea rsi, [secret_str] ; 2ème argument : "secret"
lea rdi, [input]      ; 1er argument : notre input
call strcmp
test eax, eax          ; strcmp retourne 0 si égal
jne .wrong             ; si != 0 → mauvais mot de passe
; ... code "correct" ...
.wrong:
```

Astuce RE : `call strcmp + test eax, eax + jne` = vérification de mot de passe. Breakpoint sur `strcmp` → `x/s $rsi` = mot de passe trouvé.

5. Lire l'assembleur dans GDB

```
gcc -g -O0 -o prog source.c
gdb ./prog
(gdb) break main
(gdb) run
(gdb) disassemble main      # voir l'assembleur de main
(gdb) layout asm            # vue assembleur en temps réel
(gdb) layout regs           # vue registres en temps réel
(gdb) si                    # step instruction (une instruction asm à la
fois)
(gdb) ni                    # next instruction (sans entrer dans les
fonctions)
```

6. Obtenir l'assembleur sans GDB

```
# Désassembler avec objdump
objdump -d -M intel ./prog | less

# Voir uniquement la fonction main
objdump -d -M intel ./prog | grep -A 50 "<main>:"
```

`-M intel` = syntaxe Intel (plus lisible que la syntaxe AT&T par défaut)

Récapitulatif des patterns

Code C	Pattern assembleur
<code>int x = 5</code>	<code>mov DWORD PTR [rbp-N], 5</code>
<code>if (a > b)</code>	<code>cmp + jle/jge</code>
<code>for (i=0; i<n; i++)</code>	<code>init + cmp + jge + jmp arrière</code>
<code>func(a, b)</code>	<code>mov edi, a + mov esi, b + call func</code>
<code>return x</code>	<code>mov eax, x + ret</code>
<code>tab[i]</code>	<code>[base + i*4]</code>
<code>strcmp(a, b) == 0</code>	<code>call strcmp + test eax, eax + jne</code>
<code>malloc(n)</code>	<code>mov rdi, n + call malloc (retour dans rax)</code>

Code C**Pattern assembleur**

`x ^= clé``xor [rbp-N], clé`

Leçons apprises

- Lire l'asm = reconnaître des patterns, pas mémoriser des instructions
- `cmp` + saut conditionnel = toujours un `if` ou une condition de boucle
- `jmp` vers une adresse précédente = boucle
- Les arguments de fonctions sont dans `rdi`, `rsi`, `rdx`...
- La valeur de retour est toujours dans `rax`

Prochaine leçon

Leçon 4 — Ghidra : ouvrir un vrai binaire, décompiler en pseudo-C, identifier les fonctions et structures de données.